

Project Report

Intelligent IoT-Based Smart Irrigation System

Prepared by: Abhi

1. Abstract

The Intelligent IoT-Based Smart Irrigation System is a modern, data-driven agricultural solution designed to optimize water conservation by integrating Edge hardware with advanced Machine Learning. Utilizing an ESP32 microcontroller, the system continuously monitors environmental telemetry, including ambient temperature, humidity, and soil moisture. This raw data is transmitted via a local Wi-Fi network to a Python backend server, where a Random Forest classification algorithm evaluates the conditions to make highly accurate, dynamic watering decisions. When hydration is necessary, the server triggers a physical relay and water pump. A cross-platform mobile application, built using the Flet framework, serves as a real-time monitoring dashboard, providing the user with live metrics, AI prediction states, and manual override capabilities.

2. Introduction

2.1 Problem Statement

Traditional automated irrigation systems rely on rigid, time-based scheduling. These systems are "blind" to actual environmental conditions, leading to significant water waste during periods of high humidity or rainfall, and crop dehydration during unexpected heatwaves.

2.2 Objectives

- To develop a context-aware irrigation system that responds to real-time physical telemetry rather than a fixed clock.
- To implement a Machine Learning model capable of analyzing the complex relationships between multiple weather and soil variables.
- To create a seamless, remote user interface for real-time monitoring and hardware control.

3. System Architecture

The project architecture is divided into three distinct layers, ensuring modularity, scalability, and clean data routing.

3.1 Layer 1: The Edge (Hardware)

The physical foundation of the system. It is responsible for gathering raw data from the soil and air, and executing the physical actions (pumping water) dictated by the central server.

- **Central Node:** ESP32 Microcontroller
- **Inputs:** DHT11 Digital Sensor, Analog Soil Moisture Probe
- **Outputs:** 5V Relay Module, Submersible Water Pump

3.2 Layer 2: The Backend (AI & Server)

The logic center of the architecture. It hosts the REST API to receive hardware payloads and runs the Machine Learning inference engine.

- **Server:** Python-based HTTP REST API
- **Machine Learning Engine:** Scikit-Learn Random Forest Classifier

3.3 Layer 3: The Client (User Interface)

The frontend dashboard that allows human interaction with the automated system.

- **Framework:** Python Flet (Powered by Flutter)
- **Features:** Live sensor readouts, AI status indicators, and a manual hardware override switch.

4. Hardware Specifications

- **ESP32 Microcontroller:** Chosen for its dual-core processing and built-in Wi-Fi capabilities, allowing it to act as an IoT bridge without external networking modules.
- **DHT11 Sensor:** Measures ambient atmospheric temperature and relative humidity, providing above-ground context for the AI.

- **Soil Moisture Sensor (Analog):** Measures the volumetric water content in the soil by calculating electrical resistance between its two probes.
- **5V Single-Channel Relay:** Acts as an electrically operated switch, allowing the low-voltage ESP32 (3.3V logic) to safely control the high-power water pump.
- **Water Pump:** Submersible motor responsible for the physical delivery of water to the crop.

5. Software & Machine Learning Implementation

5.1 The Random Forest Classifier

The decision-making core relies on a Random Forest algorithm. Unlike simple decision trees that are prone to overfitting, the Random Forest creates an ensemble of randomized trees to process the input array [Temperature, Humidity, Soil Moisture]. It was trained on historical agricultural datasets to output a binary classification: Class 1 (Pump ON) and Class 0 (Pump OFF). This model handles the inherent "noise" of cheap physical sensors beautifully, ensuring that a single glitchy temperature reading does not accidentally flood the crops.

5.2 Mobile Application (Flet)

The Android mobile application serves as a "thin client." It utilizes background polling to continuously fetch JSON state updates from the Python server. This ensures the dashboard remains perfectly synchronized with the hardware state in real-time, allowing the user to monitor operations from anywhere on the local network.

6. System Workflow & Data Pipeline

1. **Data Acquisition:** Every 10 seconds, the ESP32 reads analog and digital signals from the connected sensors.
2. **Transmission:** The ESP32 formats this data into a JSON payload and transmits it via HTTP POST over Wi-Fi to the Python backend.
3. **AI Inference:** The Python server extracts the telemetry and passes it into the pre-trained Random Forest model.
4. **Hardware Actuation:** The server replies to the ESP32 with the AI's binary decision. If the decision is 1, the ESP32 sends a HIGH signal to the Relay pin, closing the circuit and turning on the water pump.

5. **Client Synchronization:** The Flet Android app detects the server state change and updates the UI dashboard instantly.

7. Conclusion & Future Scope

The Intelligent IoT-Based Smart Irrigation System successfully proves the viability of replacing rigid agricultural timers with dynamic, Edge-to-Cloud Machine Learning architectures. By relying on real-time data and predictive algorithms, the system minimizes water waste and maximizes crop health.

Future Enhancements: Migrating the local Python REST API to a cloud-based asynchronous framework (e.g., FastAPI) utilizing MQTT protocols to support thousands of simultaneous sensor nodes. Integrating external weather API forecasting so the Machine Learning model can delay watering if rain is predicted within the next 24 hours.